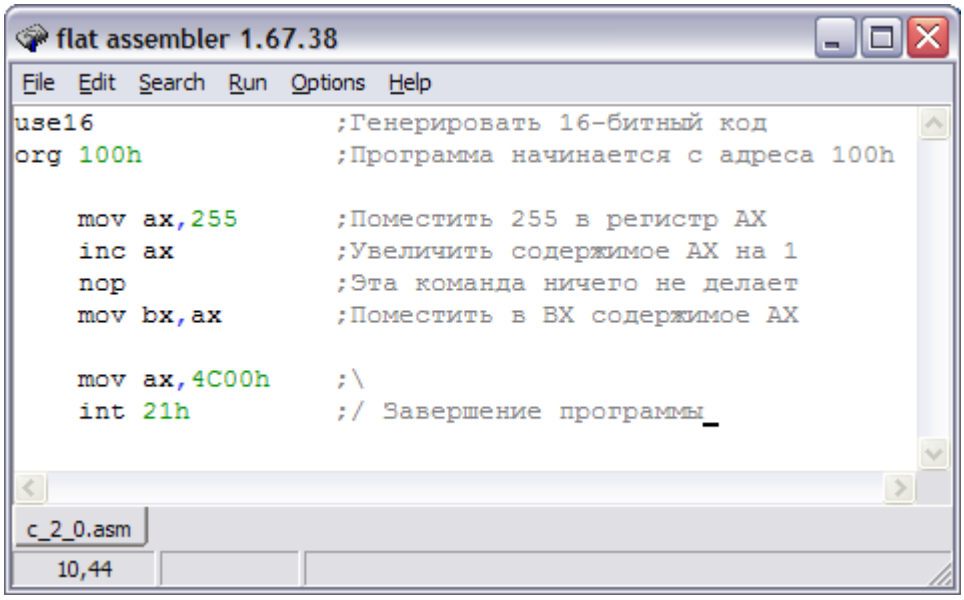
Первая программа

Для того, чтобы написать программу, нам надо запустить fasmw.exe. Откроется окошко, в которое можно смело набивать код:



В это окошко надо ввести следующее (я подробно объясню ниже, что значит каждая строчка):

```
1  use16                ;Генерировать 16-битный код
2  org 100h             ;Программа начинается с адреса 100h
3
4      mov ax,255       ;Поместить 255 в регистр AX
5      inc ax           ;Увеличить содержимое AX на 1
6      nop              ;Эта команда ничего не делает
7      mov bx,ax        ;Поместить в BX содержимое AX
8
9      mov ax,4C00h     ;\
10     int 21h          ;/ Завершение программы
```

Первая строка *«use16»* сообщает FASM'у, что нужно генерировать 16-битный код. Нам нужен именно такой для нашей первой программы. Точка с запятой — это символ комментария. Все что идет после *«;»* до конца строки игнорируется компилятором. Там можно писать все что угодно.

Вторая строка *«org 100h»* объясняет FASM'у, что следующие команды и данные будут располагаться в памяти, начиная с адреса 100h. Дело в том, что при загрузке нашей программы в память, DOS размещает в первых 256 байтах (с адресов 0000h — 00FFh) свои служебные данные. Нам эти данные изменять нежелательно.

Далее идут непосредственно команды! Программа на ассемблере состоит из команд процессора. Каждая команда обозначается мнемоникой (символическим именем). Например *«mov»*, *«inc»*, *«nop»* и т.д. После мнемоники могут идти операнды. Они отделяются одним или несколькими пробелами (или табуляцией).

Команды бывают без операндов, с одним или несколькими операндами. Если операндов больше одного, то они отделяются друг от друга запятыми.

Отступы не обязательны, но желательны — с ними код гораздо легче читать. Пустые строки игнорируются. Регистр символов значения не имеет. Можно писать большими буквами, или маленькими, или в перемешку.

Четвертая строка определяет команду «поместить число 255 в регистр AX». *«mov»* — это мнемоника команды (от английского «MOVe»). AX — первый операнд — приёмник. 255 — второй операнд — источник. Первый операнд является регистром. Второй операнд — константа 255.

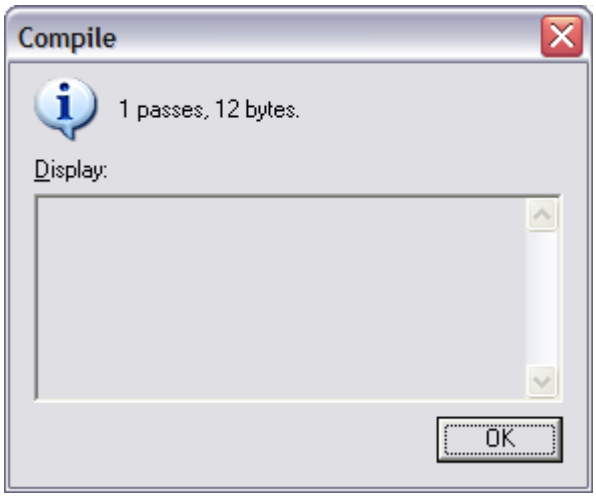
Пятая строка. Тут команда *«inc»* с одним операндом. Она заставит процессор выполнить инкремент, то есть увеличение на единицу. Единственный операнд — это регистр AX, содержимое которого и будет увеличено на 1.

Шестая строка. Команда *«nop»* — без операндов. Эта команда ничего не делает, ее назначение будет объяснено в последующих примерах.

Седьмая строка. Снова команда *«mov»*, но на этот раз оба операнда являются регистрами. Команда скопирует в BX содержимое AX.

Две последние строки — это стандартное завершение процесса в DOS. Так мы будем завершать все наши программы. Команда *«mov»* должна быть вам понятна, а про команду *«int»* я ещё расскажу, это отдельная тема.

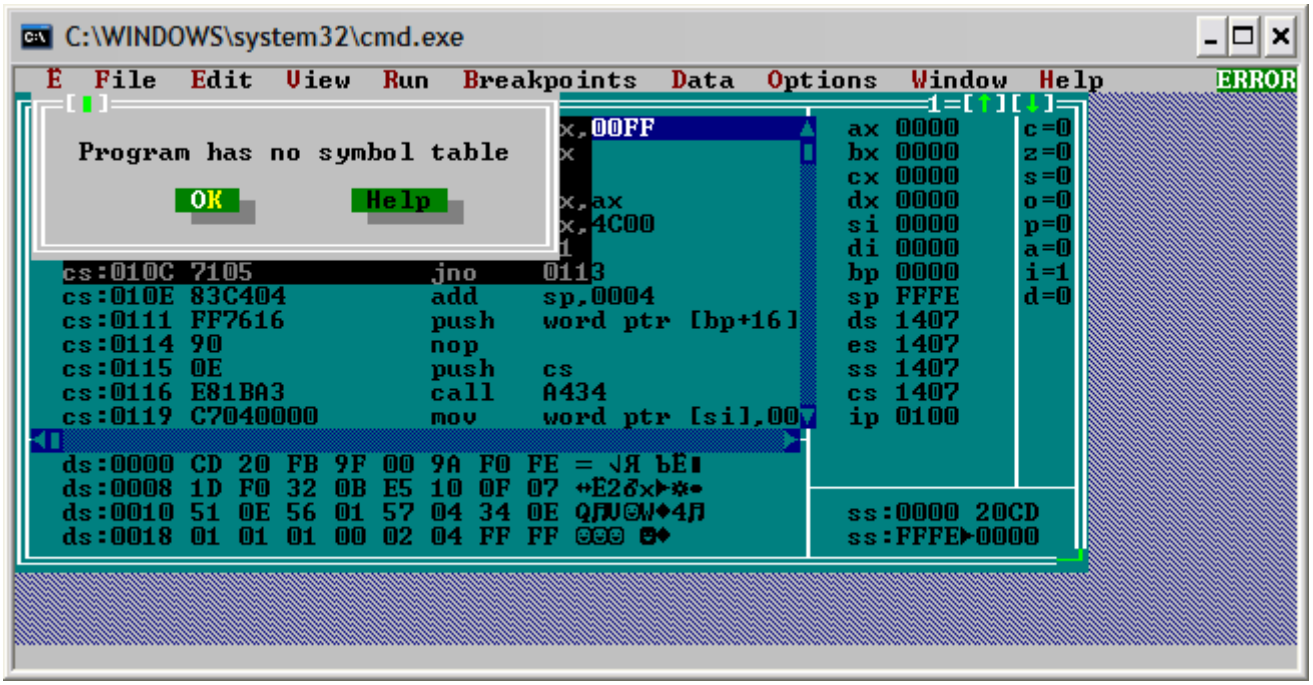
Чтобы откомпилировать программу надо выбрать меню *Run->Compile*. FASM предложит сохранить файл, если вы этого ещё не сделали, а затем скомпилирует. То есть переведет текст, набранный нами, в машинный код и сделает его программой. Файл с расширением .asm — это исходный код или исходник, обычный текстовый файл. При желании его можно открыть блокнотом)



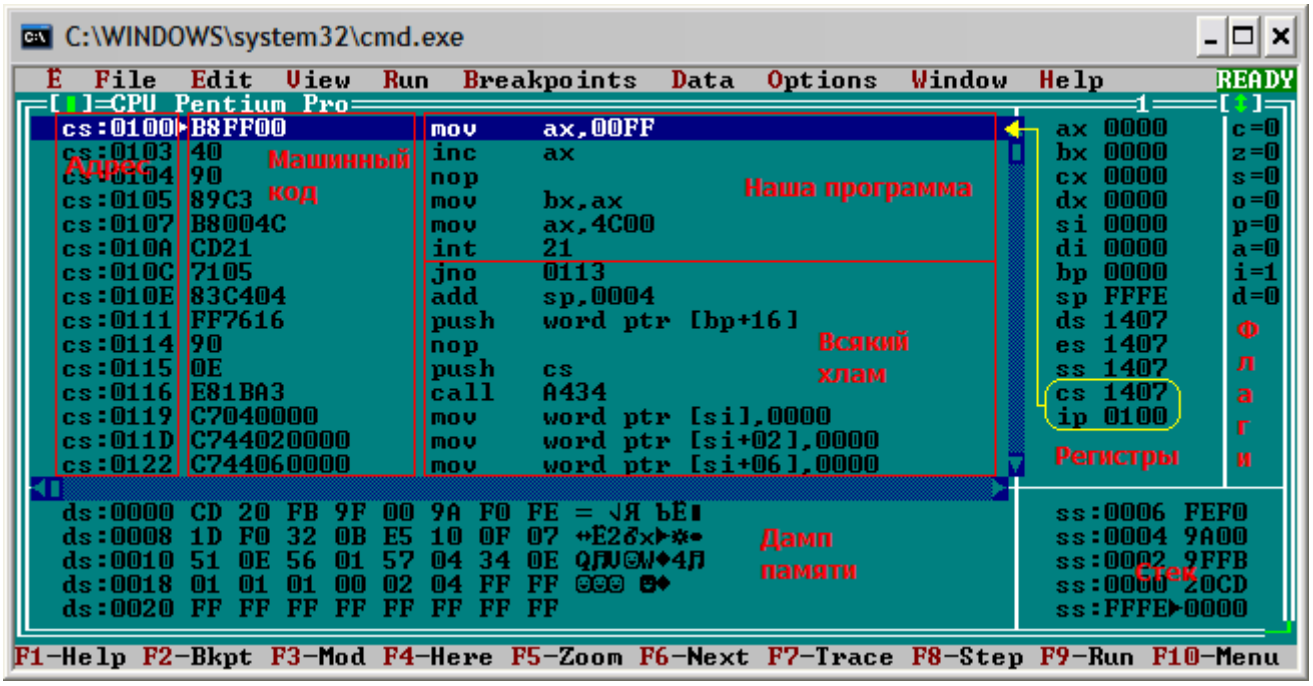
Если в коде что-то неправильно, то в этом окне вы увидите сообщение об ошибке.

Turbo Debugger

Запускаем созданный нами батник и видим такое окно:



Сообщение означает, что в исполняемом файле нет специальных данных для отладки. Но нам эти данные и не нужны, потому что программа простая и понятная. Нажимаем OK. Turbo Debugger отображает окно CPU, в котором можно увидеть, как выполняется программа.

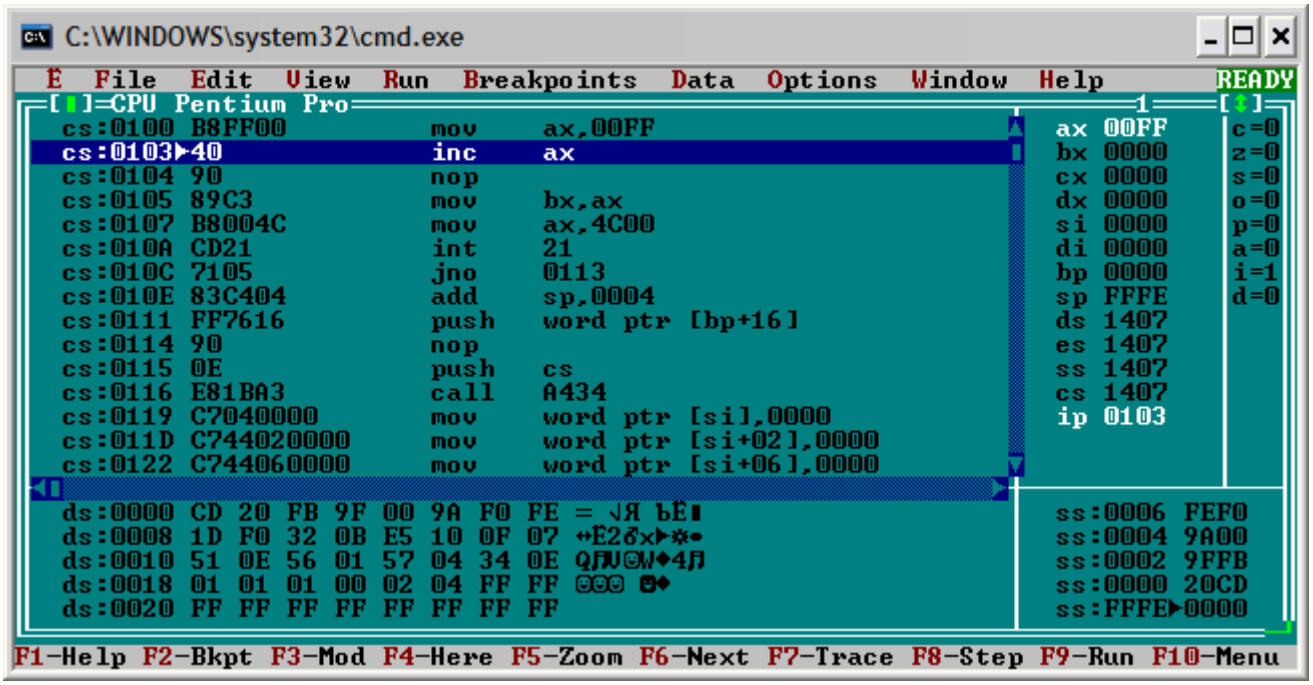


В большой области мы видим код нашей программы. Самый левый столбец — адреса, правее отображаются байты машинного кода, а ещё правее — символическое обозначение команд. Программа размещается в памяти, начиная с адреса 0100h в сегменте кода. В нашей программе всего 6 машинных команд, а за ними в памяти находится случайный мусор (точные значения неизвестны).

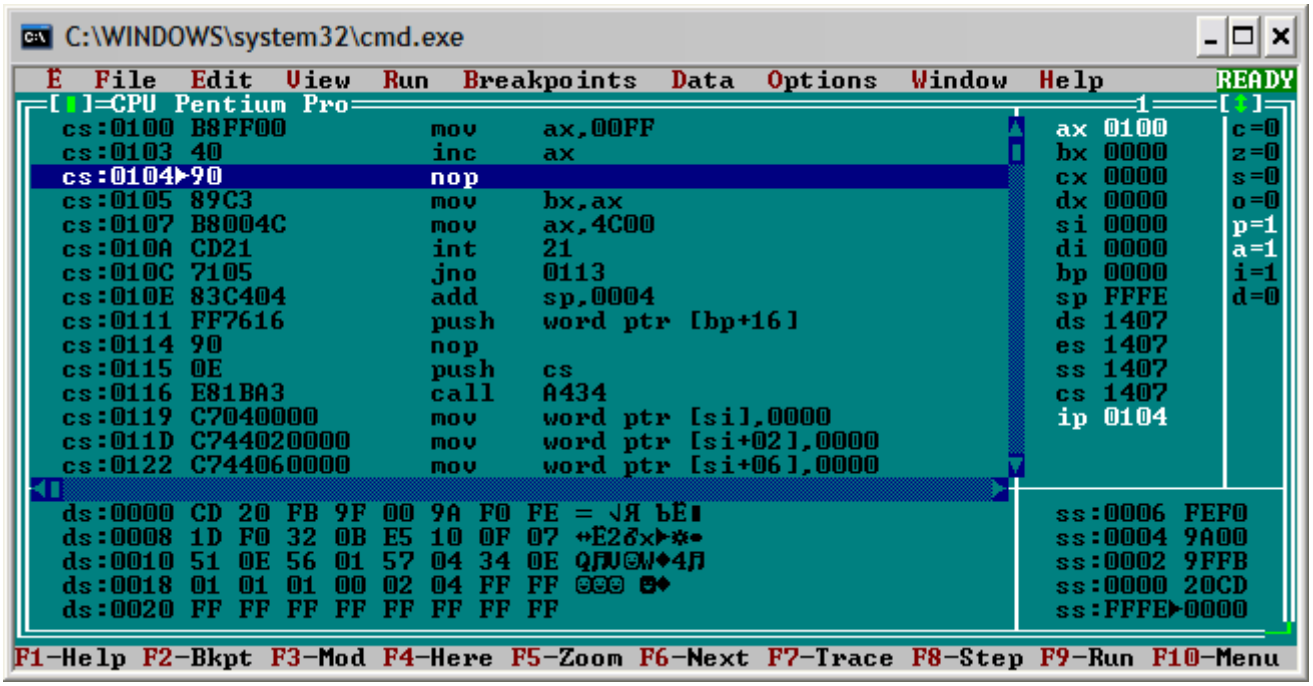
Обратите внимание, что отладчик показывает адреса и значения в шестнадцатеричном виде.

В правой части окна CPU отображаются регистры процессора и флаги. В нижней части можно увидеть дамп области памяти и стек. Стек — это специальная структура данных, с которой работают некоторые команды процессора.

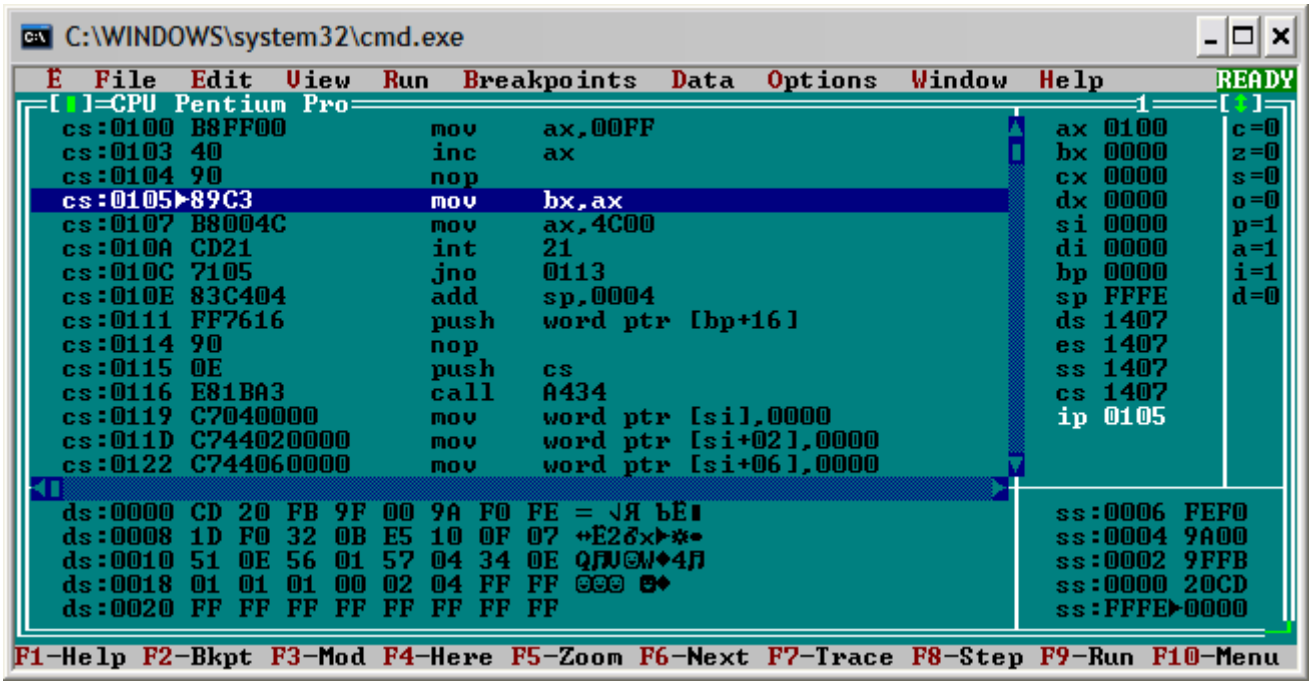
Адрес текущей машинной команды определяется регистрами CS и IP, эта команда показана выделенной строкой и стрелкой. Теперь нажмите F8, чтобы выполнить первую команду.



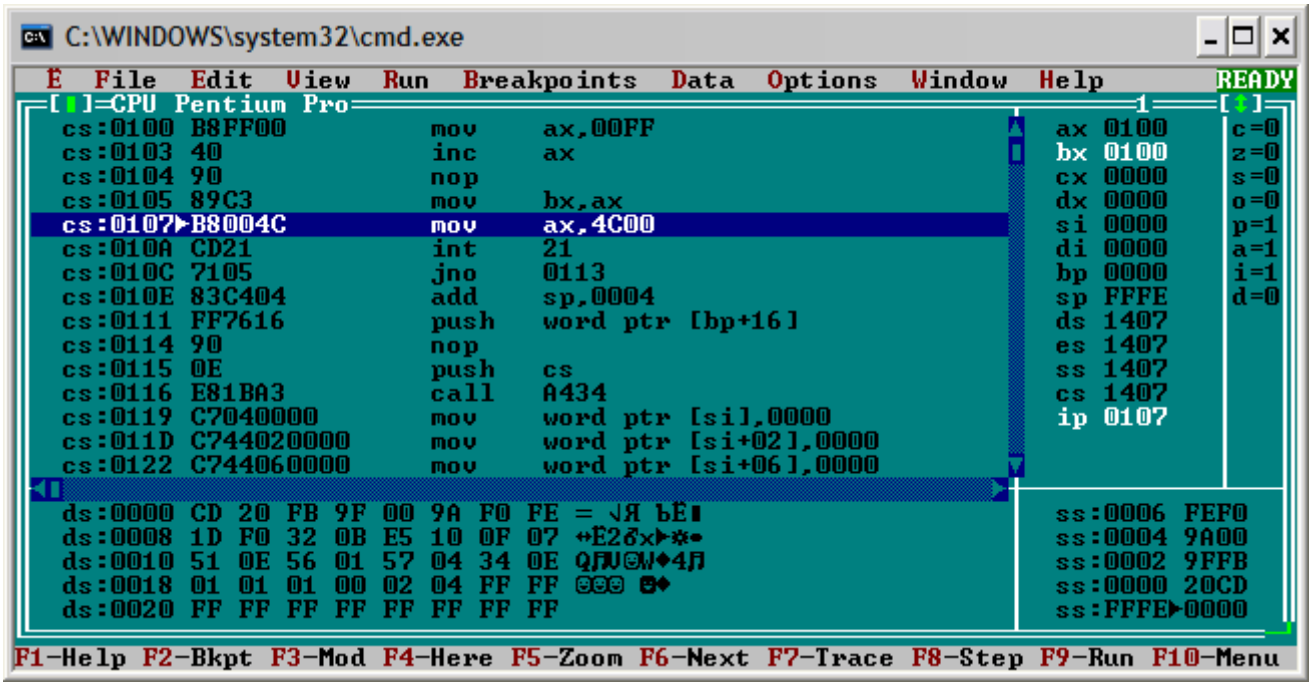
Теперь стрелка указывает на вторую команду. Изменившиеся регистры выделены белым цветом. Регистр AX теперь содержит значение 00FFh (то есть 255, чего мы и хотели от команды «mov ax,255»). Также изменилось значение регистра IP — оно увеличилось на размер выполненной машинной команды, а именно на 3. Теперь CS:IP указывает на следующую команду. Снова нажимаем F8.



Значение регистра AX увеличилось на 1 и стало равным 0100h (256). Значение IP тоже увеличилось на 1, потому что длина команды «inc ax» — 1 байт. Процессор выполняет программу последовательно, одну команду за другой. Ещё раз нажимаем F8.



Команда NOP ничего не делает. Меняется только значение IP — снова увеличивается на 1. Снова F8.



Значение BX становится равным AX. После ещё двух нажатий F8 программа завершается. Закрыть отладчик можно с помощью меню *File->Quit*.

Регистры процессора 8086

Для того, чтобы писать программы на ассемблере, нам необходимо знать, какие регистры процессора существуют и как их можно использовать. Все процессоры архитектуры x86 (даже многоядерные, большие и сложные) являются дальними потомками древнего Intel 8086 и совместимы с его архитектурой. Это значит, что программы на ассемблере 8086 будут работать и на всех современных процессорах x86.

Все внутренние регистры процессора Intel 8086 являются 16-битными:



Всего процессор содержит 12 программно-доступных регистров, а также регистр флагов (FLAGS) и указатель команд (IP).

Регистры общего назначения (РОН) AX, BX, CX и DX используются для хранения данных и выполнения различных арифметических и логических операций. Кроме того, каждый из этих регистров поделён на 2 части по 8-бит, с которыми можно работать как с 8-битными регистрами (AH, AL, BH, BL, CH, CL, DH, DL). Младшие части регистров имеют в названии букву L (от слова *Low*), а старшие H (от слова *High*). Некоторые команды неявно используют определённый регистр, например, CX может выполнять роль счетчика цикла.

Индексные регистры предназначены для хранения индексов при работе с массивами. SI (*Source Index*) содержит индекс источника, а DI (*Destination Index*) — индекс приёмника, хотя их можно использовать и как регистры общего назначения.

Регистры-указатели BP и SP используются для работы со стеком. BP (*Base Pointer*) позволяет работать с переменными в стеке. Его также можно использовать в других целях. SP (*Stack Pointer*) указывает на вершину стека. Он используется командами, которые работают со стеком. (Про стек я подробно расскажу в отдельной части учебного курса)

Сегментные регистры CS (*Code Segment*), DS (*Data Segment*), SS (*Stack Segment*) и ES (*Enhanced Segment*) предназначены для обеспечения сегментной адресации. Код находится в сегменте кода, данные — в сегменте данных, стек — в сегменте стека и есть еще дополнительный сегмент данных. Реальный физический адрес получется путём сдвига содержимого сегментного регистра на 4 бита влево и прибавления к нему смещения (относительного адреса внутри сегмента).

COM-программа всегда находится в одном сегменте, который является одновременно сегментом кода, данных и стека. При запуске COM-программы сегментные регистры будут содержать одинаковые значения.

Указатель команд IP (*Instruction Pointer*) содержит адрес команды (в сегменте кода). Напрямую изменять его содержимое нельзя, но процессор делает это сам. При выполнении обычных команд значение IP увеличивается на размер выполненной команды. Существуют также команды передачи управления, которые изменяют значение IP для осуществления переходов внутри программы.

Регистр флагов FLAGS содержит отдельные биты: флаги управления и признаки результата. Флаги управления меняют режим работы процессора:

- D (*Direction*) — флаг направления. Управляет направлением обработки строк данных: DF=0 — от младших адресов к старшим, DF=1 — от старших адресов к младшим (для специальных строковых команд).

- I (*Interrupt*) — флаг прерывания. Если значение этого бита равно 1, то прерывания разрешены, иначе — запрещены.
- T (*Trap*) — флаг трассировки. Используется отладчиком для выполнения программы по шагам.

Признаки результата устанавливаются после выполнения арифметических и логических команд:

- S (*Sign*) — знак результата, равен знаковому бита результата операции. Если равен 1, то результат — отрицательный.
- Z (*Zero*) — флаг нулевого результата. ZF=1, если результат равен нулю.
- P (*Parity*) — признак чётности результата.
- C (*Carry*) — флаг переноса. CF=1, если при сложении/вычитании возникает перенос/заём из старшего разряда. При сдвигах хранит значение выдвигаемого бита.
- A (*Auxiliary*) — флаг дополнительного переноса. Используется в операциях с упакованными двоично-десятичными числами.
- O (*Overflow*) — флаг переполнения. CF=1, если получен результат за пределами допустимого диапазона значений.

Директивы объявления данных

Практически любая программа кроме машинных команд содержит также какие-то данные. Например, числа, текстовые строчки, идентификаторы, различные ресурсы и т.д. Данные могут быть как константами, не меняющими своё значение во время выполнения программы, так и переменными, в которых хранятся всякие промежуточные результаты.

Прежде всего нужно научиться объявлять данные в программе. Для этого в ассемблере существуют директивы объявления данных.

Размер (в байтах)	**Объявление**	**Резервирование**
1	db	rb
2	dw du	rw
4	dd	rd
6	dp df	rp rf
8	dq	rq
10	dt	rt
N	file	

В учебном курсе для нас самыми полезными будут директивы *db*, *dw* и *dd*.

Синтаксис объявления данных

Объявлять данные очень просто — например, чтобы объявить байт со значением 5 достаточно написать:

```
x db 5
```

где *x* — название нашей переменной или константы, *db* — директива объявления байта, а *5* — значение. С помощью названия в программе можно будет обращаться к ячейке памяти, содержащей наш байт. Вообще, название не обязательно и можно его не писать, если оно не требуется:

```
db 5
```

Если запустить программу в отладчике Turbo Debugger, то в окне дампа можно увидеть результат работы директивы *db*:



Объявление последовательностей (массивов)

Иногда в программе требуется объявить массив, то есть несколько переменных одинакового размера, расположенных в памяти друг за другом. Например, чтобы объявить массив из 5 двухбайтных чисел можно написать:

```
array1 dw 1,2,3,4,5
```

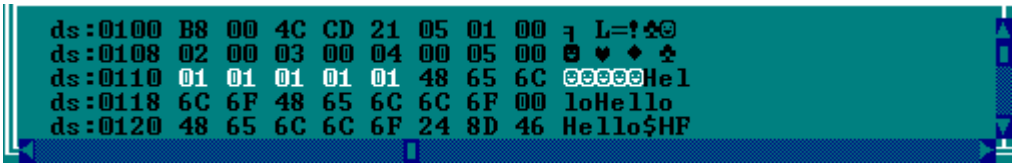
где *array1* — название массива, *1,2,3,4,5* — значения элементов. Вместо *array1* компилятор FASM будет подставлять в программу адрес начала массива, то есть адрес первого элемента.

Дамп памяти будет выглядеть следующим образом (обратите внимание, младший байт каждого слова расположен перед старшим):



Для объявления повторяющихся элементов можно использовать такую запись (объявляем массив из 5 байтов, равных 1):

```
array2 db 5 dup(1)
```



А ещё можно вот так объявить массив (догадайтесь сами, что тут получается):

```
array3 dd 4 dup(3,7,0)
```

Объявление строк

Строка представляет собой массив байтов-символов и записывается в одинарных кавычках:

```
str1 db 'Hello'
```



Для обозначения конца строки используется специальный символ. Обычно это нулевой байт, но для функций DOS используется символ ’$’.

```
str2 db 'Hello',0     ;Обычно так
str3 db 'Hello$'      ;Для DOS
```



Резервирование данных (точнее памяти для них)

Можно объявлять переменные, не имеющие определённого начального значения. Такие переменные называются неинициализированными. Например, их можно использовать в программе для хранения временного или промежуточного значения. Фактически под переменную просто резервируется место в памяти. Объявлять такие переменные можно с помощью директив *db*, *dw*, *dd*, … и знака вопроса вместо значения.

```
x1 db ?
x2 dw ?,?,?
x3 dd 10 dup(?)
```

Кроме того, FASM поддерживает специальные директивы резервирования данных. Число после директивы обозначает количество резервируемых элементов. То же самое можно объявить вот так:

```
x1 rb 1
x2 rw 3
x3 rd 10
```

С неинициализированными переменными следует быть внимательным. Не надо рассчитывать, что по умолчанию значение будет нулевым или ещё каким-то, иначе это может привести к ошибке.

Директива *file*

file — это особая директива объявления данных, которая позволяет добавить в исполняемый файл последовательность байтов из внешнего файла. Иногда это может быть очень удобно. Например, если вы хотите добавить изображение в исполняемый файл (в виде данных), или большой кусок текста, или даже код из другого файла. Директива используется следующим образом:

```
data1 file 'data.bin'        ;Добавить файл data.bin целиком.
data2 file 'data.bin':20     ;Добавить байты из файла data.bin, начиная со смещения 20.
data3 file 'data.bin':20,5   ;Добавить 5 байтов из файла data.bin, начиная со смещения 20.
```

Hello, world!

Для начала необходимо с помощью директивы *db* объявить строку, содержащую сообщение «Hello, world!». Лучше сделать это в конце программы, за последней командой, иначе процессор может принять строку за код и попытаться её выполнить.

Для вывода строки используется системная функция DOS. Чтобы напечатать строку, нужно поместить 9 в регистр AH, а в регистр DX поместить адрес строки, которая должна заканчиваться символом ‘$’. Обращение к функциям DOS осуществляется с помощью команды *int 21h*. Вот код программы:

```
1 use16                 ;Генерировать 16-битный код
2 org 100h              ;Программа начинается с адреса 100h
3
4     mov dx,hello      ;В DX адрес строки.
5     mov ah,9          ;Номер функции DOS.
6     int 21h           ;Обращение к функции DOS.
```

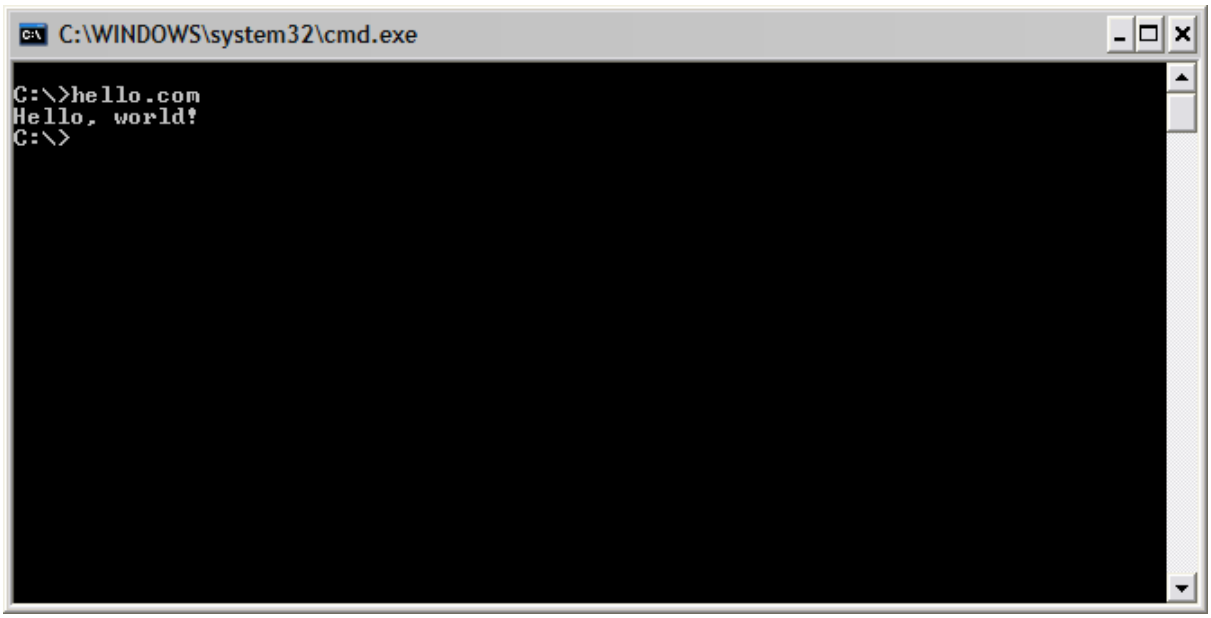
```
7
8       mov ax,4C00h     ;\
9       int 21h          ;/ Завершение программы
10  ;-------------------------------------------------------
11  hello db 'Hello, world!$'
```

В четвёртой строке FASM подставит адрес строки вместо *hello*. Не трудно догадаться, что завершение программы — это тоже функция DOS с номером 4Ch. Перед её вызовом в регистр AL помещается код завершения программы (ноль соответствует успешному завершению). Можно объединить эти две операции и сразу поместить в AX значение 4C00h.

Чтобы увидеть работу программы, надо запустить её из командной строки, иначе она печатает строку и сразу закрывается. Или можно написать простенький bat-файл для запуска:

```
1 hello.com
2 pause
```

Результат работы программы:



Если вы запустите программу в отладчике, то просмотреть выводимую строку можно, нажав Alt+F5 или выбрав в меню Turbo Debuger пункт *Window->User Screen*.

Числа со знаком и без

Числа со знаком и дополнительный код

Помимо того, что процессор работает с двоичными числами, эти числа могут быть со знаком или без знака. Если число без знака, то оно просто представляет собой результат перевода десятичного числа в двоичный вид. Все биты в таком числе являются *информационными* и оно может принимать только неотрицательные значения.

Для представления чисел со знаком используется специальное кодирование. Старший бит в этом случае обозначает знак числа. Если знаковый бит равен нулю, то число положительное, иначе — отрицательное. Понятно, что положительное число со знаком будет выглядеть точно так же, как и число без знака.

С отрицательными числами чуть сложнее. Исторически для представления отрицательных чисел в компьютерах использовались разные виды кодирования: прямой, обратный и дополнительный код. В настоящее время наиболее часто используется дополнительный код, в том числе и в процессорах x86.

Чтобы сделать из положительного числа отрицательное, необходимо проинвертировать все его биты (0 заменяем на 1, а 1 заменяем на 0) и затем к младшему разряду прибавить единицу. Например, представим -5 в дополнительном коде:



В обратную сторону переводится точно также

Синтаксис FASM

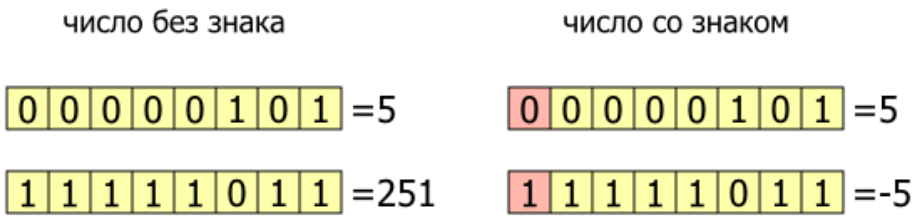
Для записи отрицательного числа в программе на ассемблере используется символ ‘-‘, например:

```
x db -5
```

Кстати, это работает и с числами в других системах счисления, и даже с символами

```
y db -25h
z db -77o
k db -101b
s db -'a'
```

Со знаковыми и беззнаковыми числами нужно быть внимательным, потому что только вы знаете, какие числа используются в вашей программе! Процессору совершенно не важно, какие данные он обрабатывает, поэтому невнимательность может привести к ошибке. Один и тот же байт может интерпретироваться по-разному, в зависимости от того со знаком число или без. Например, числу со знаком -5 соответствует число без знака 251:



Диапазоны значений чисел со знаком и без

При программировании на ассемблере (как, впрочем, и на многих других языках) необходимо учитывать ещё один важный момент. А именно — ограничение диапазона представления чисел. Например, если размер беззнаковой переменной равен 1 байт, то она может принимать всего 256 различных значений. Это означает, что мы не сможем представить с её помощью число, больше 255 (11111111_2). Для такой же переменной со знаком максимальным значением будет 127 (01111111_2), а минимальным -128 (10000000_2). Аналогично определяется диапазон для 2- и 4-байтных переменных.

Кстати, так как процессор Intel 8086 был 16-битным и обрабатывал за одну команду 16-бит, то 16-битная переменная называется *слово (word)*, а 32-битная — *двойное слово (double word, dword)*. Эти названия сохранились в ассемблере даже для 32-битных процессоров (и в WIN32 API, например). И от них же происходят названия директив *dw (Define Word)* и *dd (Define Dword)*. Ну а *db* — это *Define Byte*.

Для наглядности вот табличка диапазонов чисел:

Размер	Число без знака	Число со знаком

переменной	min	max	min	max
байт	00000000	11111111	10000000	01111111
	0	255	-128	127
слово	00000000 00000000	11111111 11111111	10000000 00000000	01111111 11111111
	0	65 535	-32 768	32 767
двойное слово	0000…0000	1111…1111	1000…0000	0111…1111
	0	4 294 967 295	-2 147 483 648	2 147 483 647
и т.д.	…	…	…	…

Если результат какой-то операции выйдет за пределы диапазона представления чисел, то случится *переполнение* и результат будет некорректным. (Например, при сложении двух положительных чисел, можно получить отрицательное число!) Поэтому нужно быть внимательным при программировании и предусмотреть обработку таких ситуаций, если они могут возникнуть.

Системы счисления

Немного теории

Прежде всего разберёмся, в чём различие между системами счисления. Любое десятичное число можно представить в таком виде:

$123_{10} = 1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0$

Индекс внизу обозначает, что это десятичное число. Цифра каждого разряда умножается на 10 в степени, равной номеру разряда, если считать с нуля справа налево. В более общем виде:

$abc_r = a \cdot r^2 + b \cdot r^1 + c \cdot r^0$,

где a, b и c — какие-то цифры, а r — основание системы счисления. Для десятичной системы r = 10, для двоичной — r = 2, для троичной r = 3 и т.д. Например, то же самое число в других системах:

$443_5 = 4 \cdot 5^2 + 4 \cdot 5^1 + 3 \cdot 5^0 = 4 \cdot 25 + 4 \cdot 5 + 3 \cdot 1 = 123_{10}$ (пятеричная система)

$173_8 = 1 \cdot 8^2 + 7 \cdot 8^1 + 3 \cdot 8^0 = 1 \cdot 64 + 7 \cdot 8 + 3 \cdot 1 = 123_{10}$ (восьмеричная система)

$1111011_2 = 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 123_{10}$(двоичная система)

Шестнадцатеричная система

В шестнадцатеричной системе для обозначения цифр больше 9 используются буквы A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. Например:

$C7_{16} = 12 \cdot 16^1 + 7 \cdot 16^0 = 12 \cdot 16 + 7 \cdot 1 = 199_{10}$

Удобство шестнадцатеричной системы в том, что в неё очень легко можно переводить двоичные числа (и в обратную сторону тоже). Четыре разряда двоичного числа (тетрада) представляются одним разрядом шестнадцатеричного. Для перевода достаточно разбить число на группы по 4 бита и заменить каждую тетраду соответствующей шестнадцатеричной цифрой.

Двоичная тетрада	Шестнадцатеричная цифра

0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Для записи одного байта требуется всего 2 шестнадцатеричные цифры:

$0101\ 1011_2 = 5B_{16}$

$0110\ 0000_2 = 60_{16}$

$1111\ 1111_2 = FF_{16}$

Восьмеричная система

Восьмеричная система также удобна для представления двоичных чисел, хотя она используется намного реже, чем шестнадцатеричная. Для быстрого перевода надо разбить двоичное число на группы по 3 разряда (триплеты или триады).

Двоичная триада	**Восьмеричная цифра**
000	0
001	1

010	2
011	3
100	4
101	5
110	6
111	7

Например: 001 110 $101_2 = 165_8$

Синтаксис ассемблера FASM

По умолчанию, число в программе воспринимается ассемблером как десятичное. Чтобы обозначить двоичное число, необходимо к нему в конце добавить символ *'b'*. Восьмеричное число обозначается аналогично с помощью символа *'o'*. Для записи шестнадцатеричного числа FASM поддерживает 3 формы записи:

- перед числом записываются символы *'0x'* (как в C/C++);
- перед числом записывается символ *'$'* (как в Pascal);
- после числа записывается символ *'h'*. Если шестнадцатеричное число начинается с буквы, необходимо добавить в начале ноль (иначе непонятно, число это или имя метки).

Этот синтаксис используется как при объявлении данных, так и в командах. Вот примеры записи чисел во всех четырёх системах:

```
mov ax,537          ;Десятичная система

mov bl,11010001b    ;Двоичная система

mov ch,57o          ;Восьмеричная система

mov dl,$C2          ;\
mov si,0x013A       ; \
mov ah,18h          ; / Шестнадцатеричная система
mov al,0FFh         ;/

mov al,FFh          ;Ошибка!
```

Сложение и вычитание

Сложение

Для сложения двух чисел предназначена команда ADD. Она работает как с числами со знаком, так и с числами без знака (это особенность дополнительного кода).

Операнды должны иметь одинаковый размер (нельзя складывать 16- и 8-битное значение). Результат помещается на место первого операнда. В общем, эти правила справедливы для большинства команд.

После выполнения команды изменяются флаги, по которым можно определить характеристики результата:

- Флаг *CF* устанавливается, если при сложении произошёл перенос из старшего разряда. Для беззнаковых чисел это будет означать, что произошло переполнение и результат получился некорректным.
- Флаг *OF* обозначает переполнение для чисел со знаком.
- Флаг *SF* равен знаковому биту результата (естественно, для чисел со знаком, а для беззнаковых он равен старшему биту и особо смысла не имеет).
- Флаг *ZF* устанавливается, если результат равен 0.
- Флаг *PF* — признак чётности, равен 1, если результат содержит нечётное число единиц.

Примеры:

```
add ax,5          ;AX = AX + 5
add dx,cx         ;DX = DX + CX
add dx,cl         ;Ошибка: разный размер операндов.
```

Вычитание

Вычитание выполняется с помощью команды SUB. Результат также помещается на место первого операнда и опять же выставляются флаги. Единственная разница в том, что происходит вычитание, а не сложение.

На самом деле вычитание в процессоре реализовано с помощью сложения. Процессор меняет знак второго операнда на противоположный, а затем складывает два числа. Если вам необходимо в программе поменять знак числа на противоположный, можно использовать команду NEG. У этой команды всего один операнд.

Примеры:

```
sub si,dx         ;SI = SI - DX
neg ax            ;AX = -AX
```

Инкремент и декремент

Очень часто в программах используется операция прибавления или вычитания единицы. Прибавление единицы называется инкрементом, а вычитание — декрементом. Для этих операций существуют специальные команды процессора: INC и DEC. Обратите внимание, что эти команды не изменяют значение флага *CF*.

Пример программы

Чтобы всё стало совсем понятно, напишем небольшую программу. Требуется вычислить значение формулы: *e=a-(b+c-1)+(-d)*. Все числа являются 8-битными целыми со знаком. Объявим их после кода и придумаем какие-нибудь значения.

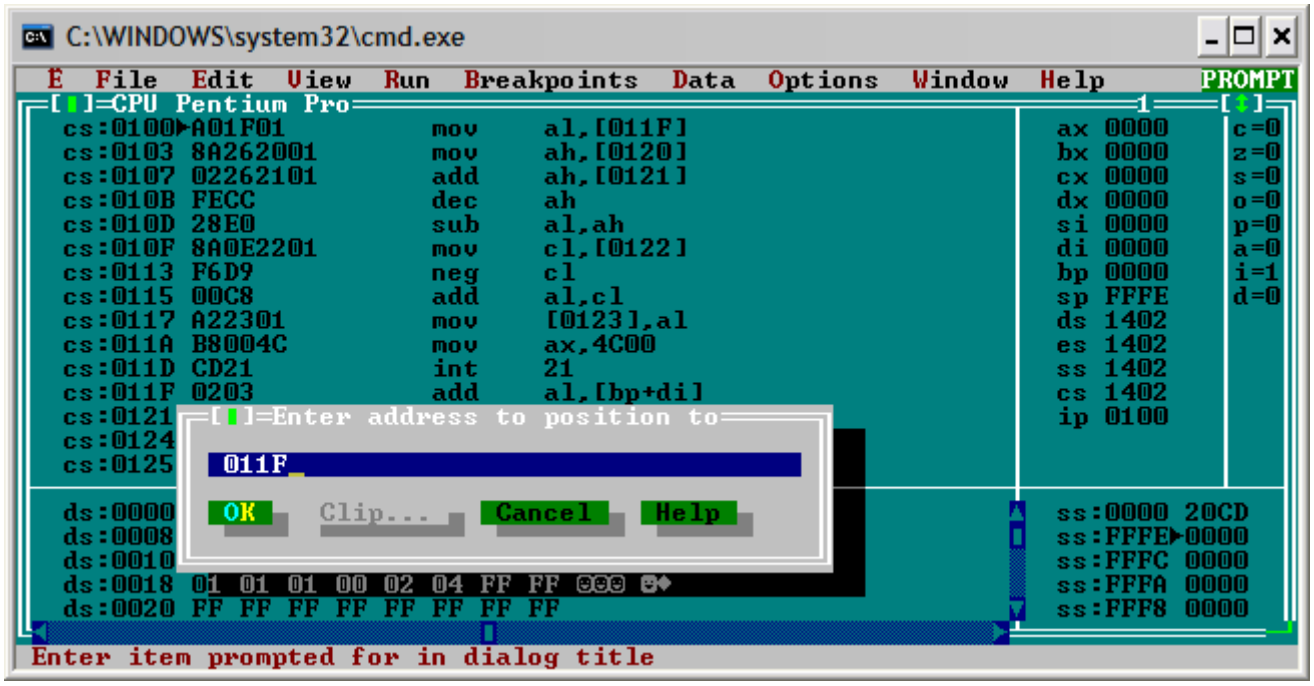
Пример:

```
1  use16               ;Генерировать 16-битный код
2  org 100h            ;Программа начинается с адреса 100h
3
4      mov al,[a]      ;Загружаем значение a в AL
5      mov ah,[b]      ;Загружаем значение b в AH
6      add ah,[c]      ;AH = AH + c = b+c
7      dec ah          ;AH = AH - 1 = b+c-1
8      sub al,ah       ;AL = AL - AH = a-(b+c-1)
9      mov cl,[d]      ;CL = d
10     neg cl          ;CL = -CL = -d
11     add al,cl       ;AL = AL + CL = a-(b+c-1)+(-d)
```

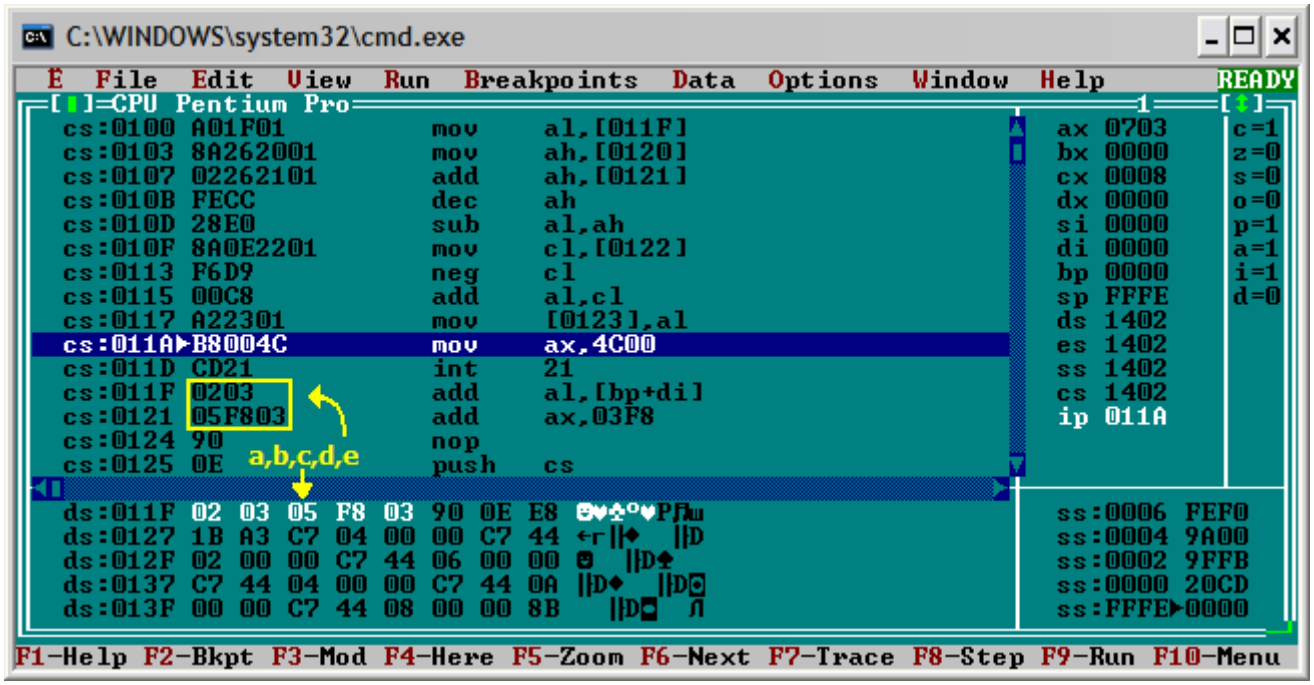
```
12      mov [e],al          ;Сохраняем результат в e
13
14      mov ax,4C00h        ;\
15      int 21h             ;/ Завершение программы
16  ;-------------------------------------------------------
17  a db 2
18  b db 3
19  c db 5
20  d db -8
21  e db ?
```

Квадратные скобки означают, что операнд находится по адресу, указанному внутри этих скобок. Так как вместо имени переменной FASM подставляет её адрес, то такая запись позволяет прочитать или записать значение переменной.

Запустив программу в Turbo Debugger, можно посмотреть её выполнение по шагам. Значения переменных можно увидеть в окне дампа памяти. Для этого нужно кликнуть правой кнопкой в этом окне и выбрать в меню пункт *Goto…*. Переменные начинаются в памяти с адреса 011Fh (этот адрес в первой команде).



В этих байтах легко угадываются наши переменные:


C:\WINDOWS\system32\cmd.exe
File Edit View Run Breakpoints Data Options Window Help READY
CPU Pentium Pro
cs:0100 A01F01 mov al,[011F]
cs:0103 8A262001 mov ah,[0120]
cs:0107 02262101 add ah,[0121]
cs:010B FECC dec ah
cs:010D 28E0 sub al,ah
cs:010F 8A0E2201 mov cl,[0122]
cs:0113 F6D9 neg cl
cs:0115 00C8 add al,cl
cs:0117 A22301 mov [0123],al
cs:011A B8004C mov ax,4C00
cs:011D CD21 int 21
cs:011F 0203 add al,[bp+di]
cs:0121 05F803 add ax,03F8
cs:0124 90 nop
cs:0125 0E push cs
a,b,c,d,e
ds:011F 02 03 05 F8 03 90 0E E8
ds:0127 1B A3 C7 04 00 00 C7 44
ds:012F 02 00 00 C7 44 06 00 00
ds:0137 C7 44 04 00 00 C7 44 0A
ds:013F 00 00 C7 44 08 00 00 8B
ax 0703
bx 0000
cx 0008
dx 0000
si 0000
di 0000
bp 0000
sp FFFE
ds 1402
es 1402
ss 1402
cs 1402
ip 011A
c=1
z=0
s=0
o=0
p=1
a=1
i=1
d=0
ss:0006 FEF0
ss:0004 9A00
ss:0002 9FFB
ss:0000 20CD
ss:FFFE 0000
F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu